

Compression and Optimal Codes

Prefix codes, code length, and Huffman's algorithm

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

**Language models are
compressors**

gzip versus a language model

enwik9 is the first 10^9 bytes of English Wikipedia — a standard compression benchmark.

compressor	compressed size
gzip	32.3% of the raw bytes
LZMA2 (7-zip)	23.0%
Chinchilla 70B + arithmetic coding	8.3%

Delétang et al., *Language Modeling Is Compression*, ICLR 2024. The comparison charges nothing for the model itself: sender and receiver must already share the 70B weights. Counted honestly, the network only pays off on large corpora.

Prediction is doing the work

The language model never sees a file format. It does one thing: assign a probability to the next token.

A coder (arithmetic coding, later this lecture) turns those probabilities into bits: high probability, few bits.

To compress is to bet on probabilities: give short names to the outcomes you expect.

This lecture builds that link from the ground up — codes, code lengths, and the optimal code.

**Give short names to common
outcomes**

A five-symbol source

Suppose a source emits

symbol	probability
A	0.40
B	0.20
C	0.15
D	0.15
E	0.10

A fixed-length binary code needs three bits per symbol because $2^2 < 5 \leq 2^3$.

Fixed-length code

One valid assignment is

$A \rightarrow 000, \quad B \rightarrow 001, \quad C \rightarrow 010,$

$D \rightarrow 011, \quad E \rightarrow 100.$

Every symbol costs three bits, so the average length is

$L = 3$ bits/symbol.

The code ignores the fact that A is four times as common as E.

Variable-length code

Try

$A \rightarrow 0, \quad B \rightarrow 111, \quad C \rightarrow 101,$

$D \rightarrow 110, \quad E \rightarrow 100.$

The expected length is

$$L = 0.40(1) + 0.20(3) + 0.15(3) + 0.15(3) + 0.10(3).$$

$$L = 2.20 \text{ bits/symbol.}$$

Encode one message

Message: A B A E

Fixed-length code:

000 001 000 100 — 12 bits.

Variable-length code:

0 111 0 100 — 8 bits.

The decoder must be able to determine where one codeword ends and the next begins. The code structure must make those boundaries recoverable from the bit stream.

An ambiguous code

Suppose

$A \rightarrow 0, \quad B \rightarrow 01, \quad C \rightarrow 10.$

The bit string 010 can be decoded as

1. $A \ C = 0 \mid 10$, or
2. $B \ A = 01 \mid 0.$

The problem is that the codeword for A is a prefix of the codeword for B.

Prefix-free codes

A code is **prefix-free** if no codeword is the prefix of another codeword.

Then decoding can proceed from left to right without separators.

Prefix-free is sufficient, but not necessary, for unique decoding. For example, $A \rightarrow 0, B \rightarrow 01$ is uniquely decodable but not prefix-free; prefix-free codes are preferred because they decode immediately from left to right.

In a binary tree:

1. each left edge writes 0,
2. each right edge writes 1,
3. symbols occupy leaves only.

V VISUAL



11

Learning outcomes

By the end of this lecture you will be able to:

1. distinguish fixed-length, uniquely decodable, and prefix-free codes,
2. relate probability to ideal code length — $\log_2 p$, with Kraft as the feasibility check,
3. state Shannon's source coding theorem,
4. construct  a Huffman tree by repeated merging,
5. compare expected code length with entropy, on a worked example and on real text,
6. contrast compression with error-correcting codes,
7. and connect probabilistic prediction with compression.

Code length follows surprise

Ideal length

L22 assigned surprise

$$I(x) = -\log_2 p(x).$$

This suggests an ideal code length

$$\ell(x) = -\log_2 p(x).$$

Examples:

$$p = \frac{1}{2} \rightarrow \ell = 1, \quad p = \frac{1}{4} \rightarrow \ell = 2, \quad p = \frac{1}{8} \rightarrow \ell = 3.$$

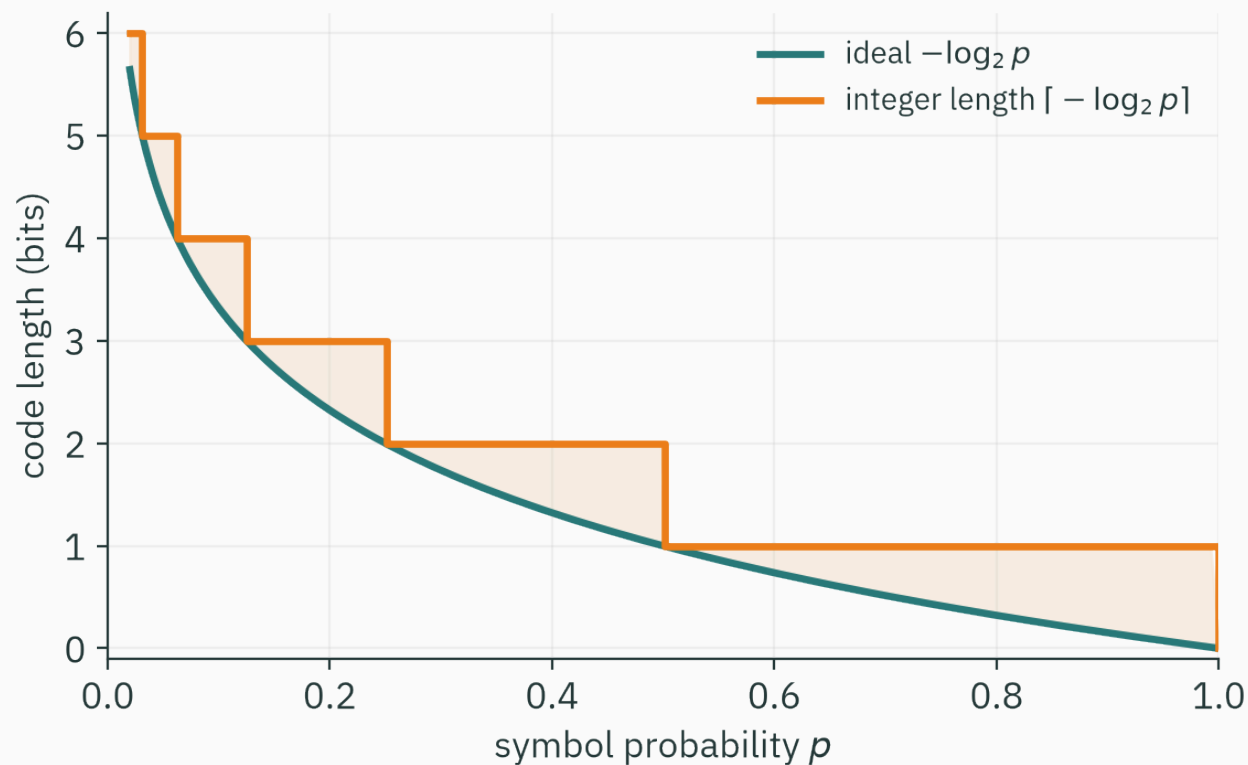
Ideal lengths need not be integers

For $p = 0.40$,

$-\log_2 0.40 \approx 1.322$ bits.

A single binary codeword cannot contain 1.322 bits.

Integer code lengths introduce a rounding cost. Coding long blocks or using arithmetic coding can approach fractional average lengths.



Choosing $\ell_i = \lceil -\log_2 p_i \rceil$ adds less than one bit to each ideal length.

The Kraft inequality

Integer lengths $\ell_1, \dots, \ell_K$ can form a binary prefix code if and only if

$$\sum_{i=1}^K 2^{-\ell_i} \leq 1.$$

Each codeword at depth ℓ_i occupies a fraction $2^{-\ell_i}$ of the leaves at a common deeper level.

Kraft check: a valid set of lengths

For lengths $(1, 3, 3, 3, 3)$,

$$2^{-1} + 4 \times 2^{-3} = \frac{1}{2} + \frac{4}{8} = 1.$$

The tree is exactly filled. This is the length pattern used for the five-symbol variable code.

Kraft check: an impossible set

For lengths $(1, 2, 2, 2)$,

$$2^{-1} + 3 \times 2^{-2} = \frac{1}{2} + \frac{3}{4} = 1.25 > 1.$$

There is not enough room in a binary tree for one one-bit leaf and three two-bit leaves.

Expected code length

For probabilities p_i and lengths ℓ_i ,

$$L = \sum_i p_i \ell_i.$$

Entropy is the corresponding average ideal length:

$$H(X) = \sum_i p_i (-\log_2 p_i).$$

For any binary prefix code,

$$L \geq H(X).$$

We use this bound today and prove the size of the gap in L24 (Gibbs' inequality).

Shannon lengths give an upper bound

Choose

$$\ell_i = \lceil -\log_2 p_i \rceil.$$

Since $\lceil a \rceil < a + 1$,

$$\ell_i \leftarrow \log_2 p_i + 1.$$

Average both sides:

$$H(X) \leq L < H(X) + 1.$$

Shannon's source coding theorem

Coding one symbol at a time can waste up to one bit on rounding. Encoding long blocks removes even that.

Source coding theorem (stated): n i.i.d. symbols from X can be encoded in about $nH(X)$ bits with error probability $\rightarrow 0$ as $n \rightarrow \infty$; below $H(X)$ bits per symbol, errors are unavoidable.

Entropy is not just a lower bound — it is the exact price of description. We state the theorem and make it plausible; MacKay Ch 4 proves it.

Why entropy is the boundary

Flip a coin with $p(\text{heads}) = 0.9$, $n = 1000$ times. Here $H \approx 0.469$ bits/flip.

Almost every sequence that actually occurs has close to 900 heads. These *typical* sequences number about

$$2^{nH} = 2^{469},$$

a vanishing fraction of the 2^{1000} possible strings.

Give each typical sequence a serial number: about 469 bits describe 1000 flips.

Counting typical sequences is the whole idea; making “almost never” precise is the work of the proof.

Which set of binary code lengths can satisfy the Kraft inequality?

A. (1, 1, 2)

B. (1, 2, 3, 3)

C. (1, 2, 2, 2)

D. (1, 1, 1)

Correct: B — (1, 2, 3, 3)

Why: $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{8} = 1$. Every other option has Kraft sum greater than one.

**Merge the two least frequent
symbols**

The greedy rule

Huffman's algorithm operates on symbol weights:

1. take the two smallest weights,
2. merge them into one node whose weight is their sum,
3. return the merged node to the collection,
4. repeat until one root remains,
5. label left and right edges 0 and 1.

Less frequent symbols are pushed deeper in the tree.

Why merge the two smallest?

In an optimal full binary tree, two deepest leaves can be made siblings.

Placing the two least probable symbols at those deepest positions cannot increase expected length.

Merge those siblings into a single pseudo-symbol. The remaining problem has one fewer symbol and the same form.

This exchange argument plus induction proves optimality.

Counts in “MATHEMATICS”

M A T H E M A T I C S contains 11 letters.

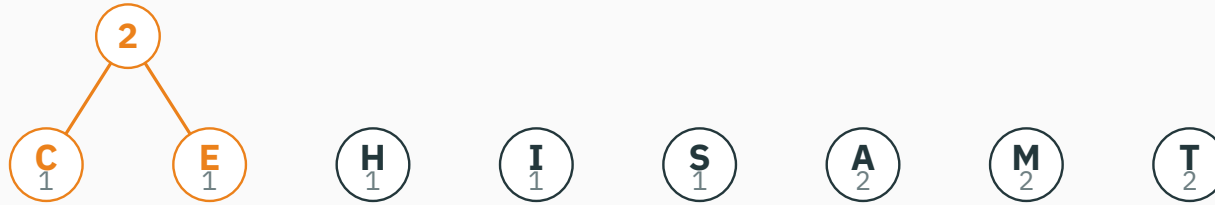
A	2	M	2
T	2	C	1
E	1	H	1
I	1	S	1

There are eight distinct symbols, so a fixed code needs three bits per letter.

★ Merge 1: C and E

D DERIVATION

The smallest weights are the five letters of count 1 — a tie. Ties can be broken arbitrarily; every choice gives the same expected length. Fix one order and take C and E first.

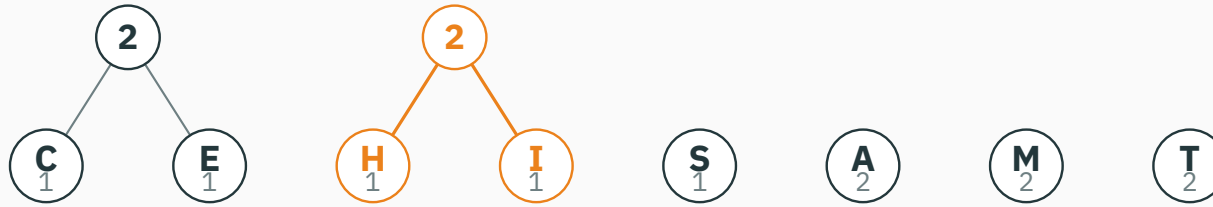


Pool: CE 2 and letters H 1, I 1, S 1, A 2, M 2, T 2.

★ Merge 2: H and I

D DERIVATION

Two of the three remaining count-1 letters merge next.



Pool: S 1 and CE 2, HI 2, A 2, M 2, T 2.

★ Merge 3: S and A

D DERIVATION

S is the only weight left at 1; its partner is any of the weight-2 nodes. Take the letter A.

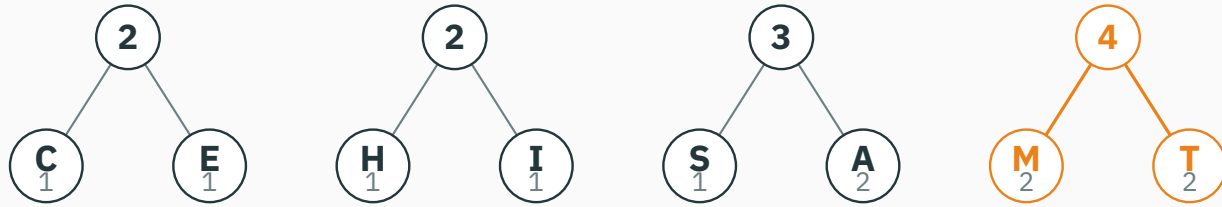


Pool: SA 3 and CE 2, HI 2, M 2, T 2.

★ Merge 4: M and T

D DERIVATION

The two smallest weights are now both 2.

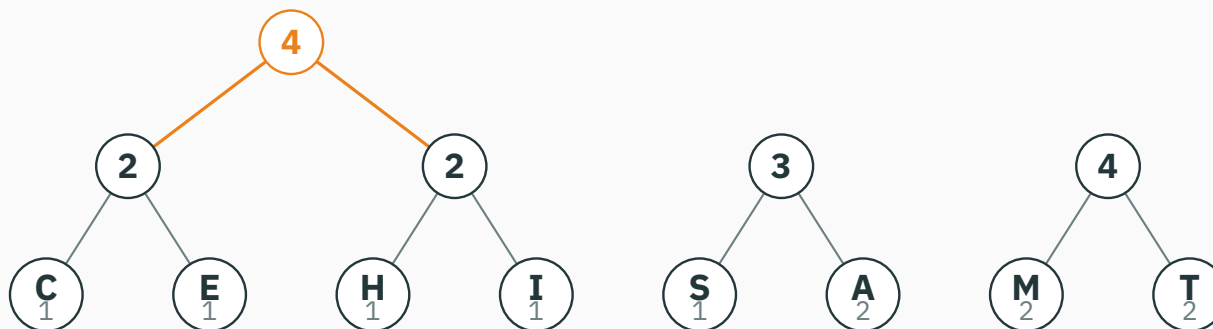


Pool: CE 2, HI 2, SA 3, MT 4.

★ Merge 5: CE and HI

D DERIVATION

The two smallest weights are the merged pairs CE and HI. Merged nodes re-enter the pool like any symbol.

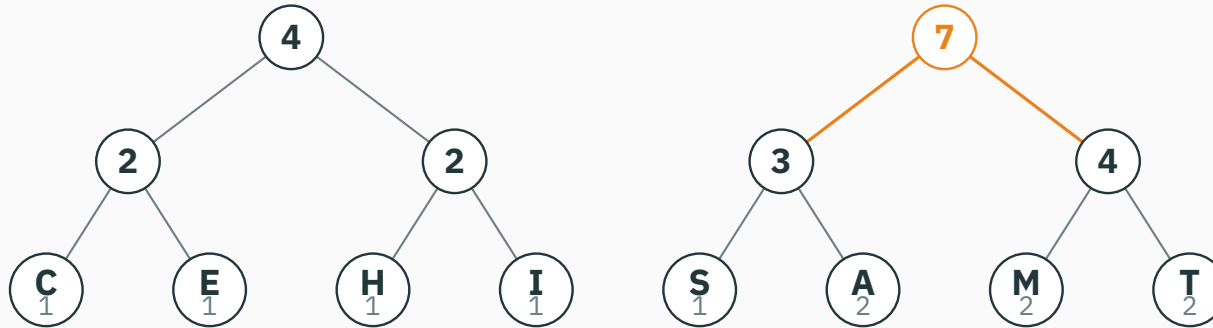


Pool: SA 3, CEHI 4, MT 4.

★ Merge 6: SA and MT

D DERIVATION

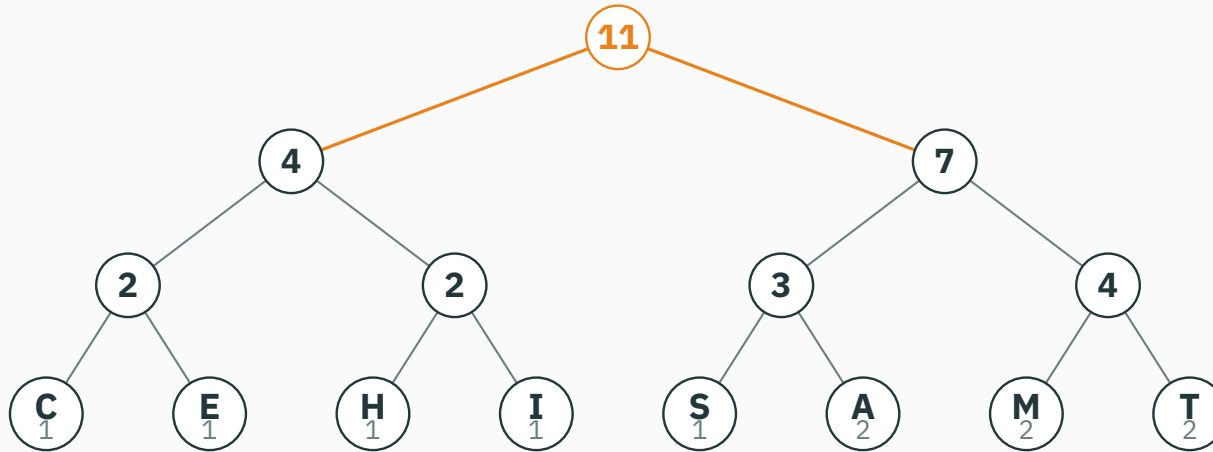
Take the 3 and one of the tied 4s.



Pool: CEHI 4, SAMT 7.

★ Merge 7: the root

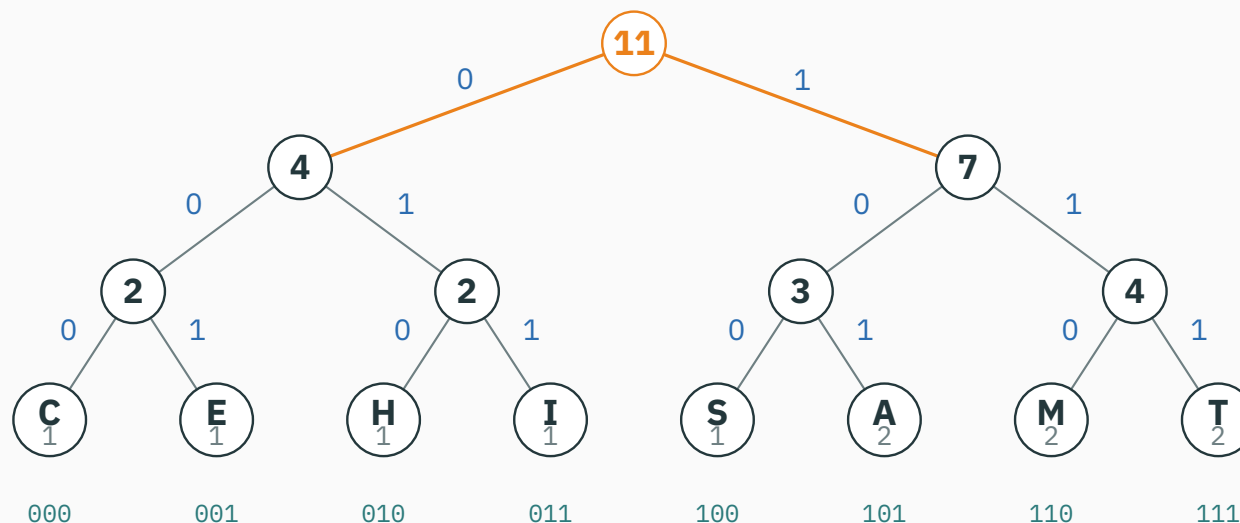
The last merge joins everything: weight $4 + 7 = 11$, the total letter count.



Seven merges for eight symbols — each merge removes one node from the pool.

The resulting tree

Label left edges 0 and right edges 1; read each codeword from root to leaf.



Here every letter happens to receive length three. Huffman cannot improve on the fixed-length code for this short frequency table.

Verify the encoded word

Reading the tree gives

$A \rightarrow 101$, $C \rightarrow 000$, $E \rightarrow 001$, $H \rightarrow 010$,

$I \rightarrow 011$, $M \rightarrow 110$, $S \rightarrow 100$, $T \rightarrow 111$.

Every codeword has length three, so “MATHEMATICS” uses

$11 \times 3 = 33$ bits.

The example is still useful: an optimal code need not beat fixed length on every finite sample.

★ Expected length versus entropy

D DERIVATION

The total bit count can be read off the build: each merge pushes its letters one level deeper, so total bits = sum of merged weights,

$$2 + 2 + 3 + 4 + 4 + 7 + 11 = 33, \quad L = \frac{33}{11} = 3.000 \text{ bits/letter.}$$

The empirical entropy of the letter counts is

$$H = -3 \cdot \frac{2}{11} \log_2 \frac{2}{11} - 5 \cdot \frac{1}{11} \log_2 \frac{1}{11} \approx 2.914 \text{ bits/letter.}$$

The bound holds:

$$H \approx 2.914 \leq L_{\text{Huffman}} = 3.000 < H + 1.$$

A source with a useful imbalance

Return to probabilities

$(0.40, 0.20, 0.15, 0.15, 0.10)$.

The merges are

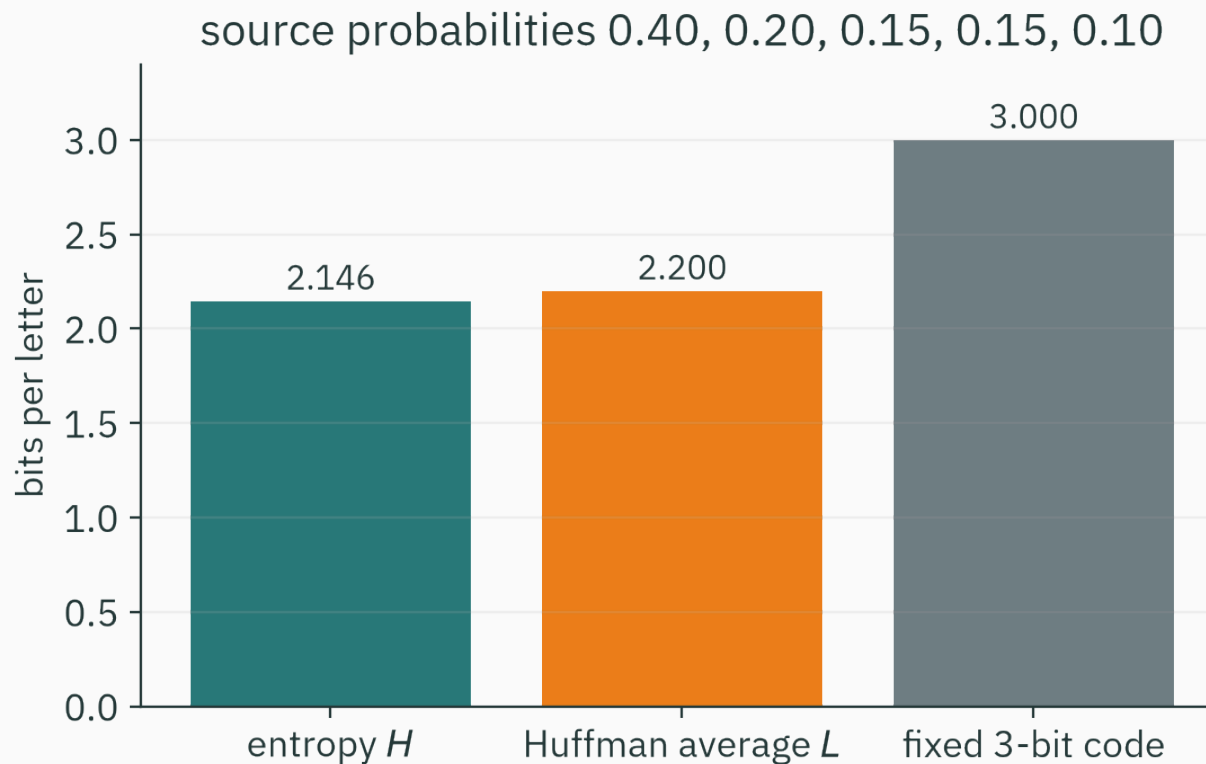
$0.10 + 0.15 \rightarrow 0.25, \quad 0.15 + 0.20 \rightarrow 0.35,$

$0.25 + 0.35 \rightarrow 0.60, \quad 0.40 + 0.60 \rightarrow 1.00.$

A alone stays shallow: lengths $(1, 3, 3, 3, 3)$, one assignment

$A \rightarrow 0, B \rightarrow 111, C \rightarrow 101, D \rightarrow 110, E \rightarrow 100.$

Compare entropy and lengths



$$H = 2.146 \leq L_{\text{Huffman}} = 2.200 < 3 = L_{\text{fixed}}.$$

L22 estimated letter probabilities from this course's plan file: 22560 letters, alphabet of 26, $\hat{H} \approx 4.238$ bits/letter. Build Huffman on those counts:



Huffman lands within 0.03 bits of the entropy: e ($\hat{p} = 0.110$) gets 3 bits, j ($\hat{p} \approx 0.002$) gets 9.

Huffman is optimal within a class

Huffman minimizes expected length among binary prefix codes for a known symbol distribution.

It does not mean:

1. the encoded file is always shorter after storing the tree,
2. one symbol at a time reaches entropy exactly,
3. the assumed probabilities match future data,
4. context has been used.

Those are separate modeling and engineering questions.

A source has counts $(5, 2, 1)$. Which code lengths does Huffman produce?

A. $(1, 2, 2)$

B. $(1, 1, 2)$

C. $(2, 2, 2)$

D. $(1, 2, 3)$

Correct: A — $(1, 2, 2)$

Why: Merge $1 + 2 \rightarrow 3$, then $3 + 5 \rightarrow 8$: the two rare symbols become depth-2 siblings. $L = (5 \cdot 1 + 2 \cdot 2 + 1 \cdot 2)/8 = 1.375$ bits versus $H \approx 1.30$. Option $(1, 1, 2)$ fails Kraft: $\frac{1}{2} + \frac{1}{2} + \frac{1}{4} > 1$.

A small compressor

A priority queue implements the merges

```
import heapq, itertools, collections

counts = collections.Counter("MATHEMATICS")
serial = itertools.count()
heap = [(count, next(serial), symbol)
        for symbol, count in sorted(counts.items())]
heapq.heapify(heap)
while len(heap) > 1:
    wa, _, a = heapq.heappop(heap)
    wb, _, b = heapq.heappop(heap)
    heapq.heappush(heap, (wa+wb, next(serial), (a, b)))
tree = heap[0][2]
```

The serial number breaks ties deterministically; sorting first reproduces the merge order we drew.

Traverse the tree to assign codes

```
def make_codes(node, prefix="", out=None):  
    out = {} if out is None else out  
    if isinstance(node, str):  
        out[node] = prefix or "0"  
    else:  
        left, right = node  
        make_codes(left, prefix + "0", out)  
        make_codes(right, prefix + "1", out)  
    return out
```

Encode by table lookup

```
def encode(text, code):  
    return "".join(code[ch] for ch in text)  
  
bits = encode("MATHEMATICS", code)  
print(bits[:12])    # 110101111010...  
print(len(bits))    # 33
```

Real implementations pack bits into bytes; a Python string of '0' and '1' is only a teaching representation.

Decode by walking the tree

```
def decode(bits, tree):
    output, node = [], tree
    for bit in bits:
        node = node[0] if bit == "0" else node[1]
        if isinstance(node, str):
            output.append(node)
            node = tree
    assert node is tree      # no unfinished codeword
    return "".join(output)
```

What must be stored?

A standalone compressed file needs enough metadata to reconstruct the code:

1. the tree or canonical code lengths,
2. the number of valid data bits in the final byte,
3. the original alphabet or symbol ordering,
4. and often checksums or format information.

For short messages, this overhead can exceed the saved bits.

The decoder does not need the exact tree shape if it receives the code length of each symbol.

Canonical Huffman coding assigns codewords deterministically from sorted lengths.

Benefits:

1. compact header,
2. fast table-based decoding,
3. reproducible code assignment.

Many file formats use this representation.

**Better probabilities give
shorter descriptions**

Code with the wrong frequency model

If a compressor expects distribution q but data come from p , it chooses lengths near

$$-\log_2 q(x).$$

Its average cost under the true source is

$$\mathbb{E}_{[X \sim p]}[-\log_2 q(X)].$$

This is cross-entropy. L24 will show exactly how much larger it is than $H(p)$.

Independent-letter model

L22 estimated letter frequencies without context.

Such a model assigns one probability to each letter regardless of its neighbors:

$$q(x_1, \dots, x_n) = \prod_{t=1}^n q(x_t).$$

It cannot exploit that u is likely after q, or that common words recur.

Conditional prediction

A context model uses

$$q(x_1, \dots, x_n) = \prod_{t=1}^n q(x_t \mid x_{<t}).$$

If the context makes the next symbol predictable, $q(x_t \mid x_{<t})$ is larger and

$$-\log_2 q(x_t \mid x_{<t})$$

is shorter.

Compression improves when prediction improves.

Huffman assigns an integer number of bits to each symbol. Arithmetic coding represents an entire sequence as an interval.

It can approach the fractional sequence length

$$-\log_2 q(x_1, \dots, x_n)$$

much more closely, especially when some probabilities are near one.

Modern learned compressors combine a probability model with an arithmetic or range coder.

Language models report coding cost

A language model outputs a probability distribution for the next token.

For observed tokens $x_1, \dots, x_n$, its total ideal code length is

$$-\sum_{t=1}^n \log_2 q(x_t \mid x_{<t}).$$

Divide by n to obtain bits per token. Exponentiate to obtain perplexity in L24.

This resolves the opening comparison: Chinchilla's 8.3% on `enwik9` is a measurement of its next-token predictions, turned into bits by an arithmetic coder. `gzip`'s dictionary model predicts worse, so it pays more bits.

**Compression and error
correction solve different
problems**

Compression removes predictable bits

Source coding uses probability to reduce average description length.

Channel coding adds structured redundancy so corrupted bits can be detected or corrected.

The two goals point in opposite directions:

1. compress before transmission,
2. then add carefully designed redundancy for the channel.

Repetition code

Encode one bit as three:

$0 \rightarrow 000, \quad 1 \rightarrow 111.$

If at most one bit flips, majority vote recovers the original:

$111 \rightarrow 101 \rightarrow \text{decode as } 1.$

The price is rate $\frac{1}{3}$: three transmitted bits per information bit.

Hamming(7,4): three parity bits protect four data bits

Send (d_1, d_2, d_3, d_4) together with

$$p_1 = d_1 \oplus d_2 \oplus d_4, \quad p_2 = d_1 \oplus d_3 \oplus d_4, \quad p_3 = d_2 \oplus d_3 \oplus d_4.$$

Each of the seven possible single-bit flips makes a distinct subset of the three checks fail, so the failure pattern points at the flipped position. Flip it back.

Rate $\frac{4}{7} \approx 0.57$ — better than $\frac{1}{3}$, and still every single error per block is corrected.

Repetition versus Hamming at 10% noise

Simulate a channel that flips each transmitted bit with probability $f = 0.1$:

code	rate	bit error after decoding
none	1	0.100
repetition R3	1/3	0.028
Hamming(7,4)	4/7	≈ 0.07

R3 reaches the lowest error but pays three transmitted bits per data bit. Hamming protects almost twice as much data per transmitted bit. MacKay Ch 1 opens with exactly this trade-off.

binary symmetric channel, flip probability $f = 0.1$

original



MFAI

received, no code (rate 1)
9.8% of bits wrong



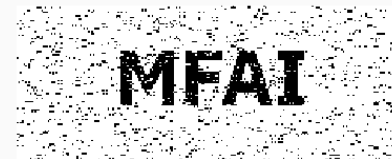
MFAI

repetition R3 (rate 1/3)
2.7% of bits wrong



MFAI

Hamming(7,4) (rate 4/7)
6.6% of bits wrong



MFAI

Every panel decodes the same bitmap after the same noisy channel (simulated, seed fixed).

A limit also exists for noisy channels

★ OPTIONAL

Shannon's channel coding theorem states that reliable communication is possible below a channel's capacity and impossible above it, under precise assumptions.

Entropy governs source description; mutual information and capacity govern transmission.

We will use mutual information only briefly in L24.

From entropy to an actual code

Frequent symbols receive short codewords; Huffman's greedy merges produce the optimal binary prefix tree.

1. Prefix-free codes decode without separators.
2. Kraft's inequality characterizes feasible prefix-code lengths.
3. Ideal length is $-\log_2 p$; entropy is the average ideal length, and the source coding theorem makes it the exact price.
4. Huffman coding minimizes expected length among binary prefix codes.
5. For a source distribution, $H \leq L_{\text{Huffman}} < H + 1$.
6. A better conditional predictor gives a shorter description.

Practice

1. Check Kraft's inequality for lengths $(2, 2, 2, 2)$ and $(1, 3, 3, 3)$.
2. Build a Huffman code for weights $(8, 4, 2, 1, 1)$.
3. Compute the expected length of codes $A : 0, B : 10, C : 110, D : 111$ under probabilities $(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$.
4. Explain why storing a Huffman tree can make a tiny file larger.

1. The Kraft sums are 1 and $\frac{1}{2} + \frac{3}{8} = \frac{7}{8}$; both are feasible.
2. One optimal code has lengths (1, 2, 3, 4, 4); the two weight-1 symbols are deepest siblings.
3. $L = \frac{1}{2}(1) + \frac{1}{4}(2) + \frac{1}{8}(3) + \frac{1}{8}(3) = 1.75$ bits/symbol, equal to the entropy.
4. The tree or length table, padding, alphabet, and format metadata create overhead independent of message length.

Next: the cost of a wrong model

Entropy is the average cost under the true distribution. A model usually supplies a different distribution.

Next lecture derives

$$H(p, q) = H(p) + D_{\text{KL}}(p \parallel q).$$

The excess code length becomes KL divergence, and minimizing cross-entropy becomes maximum likelihood.

Reading: MacKay, *Information Theory, Inference, and Learning Algorithms* — Ch 5 (Huffman), Ch 4 (source coding theorem), Ch 1 (repetition vs Hamming).
Delétang et al., *Language Modeling Is Compression*, ICLR 2024.

A good predictor and a good compressor are the same object.